UNIVERSIDAD POLITÈCNICA DE ATLAUTLA

MATERIA: FUNDAMENTOS DE INTELIGENCIA ARTIFICIAL

SECCIÒN: 4

2.3 Algoritmos de Bùsqueda Ciega

DOCENTE: JAVIER GARCÌA LARA

ALUMNA: ALEXANDRA BERNAL NÀJERA

GRUPO: 171

Asistente de Inteligencia Artificial (Google Gemini).

INTRODUCCIÓN Y MARCO TEÓRICO

Los algoritmos de búsqueda ciega (o no informada) exploran el espacio de estados sin contar con información adicional sobre cuán cerca se encuentra la meta, basándose únicamente en la definición del problema.

En esta actividad se implementan y comparan dos estrategias fundamentales para la resolución de un laberinto representado mediante un grafo:

1. BFS (Breadth-First Search / Búsqueda en Anchura): Explora el grafo nivel por nivel utilizando una estructura de datos tipo Cola (FIFO). Garantiza encontrar siempre la ruta más corta en grafos con costos uniformes, pero consume más memoria.
2. DFS (Depth-First Search / Búsqueda en Profundidad): Explora tan hondo como sea posible por cada rama antes de retroceder, empleando una Pila (LIFO) o recursividad. Utiliza menos memoria, pero no garantiza encontrar la ruta óptima y puede quedar atrapado en ramas infinitas si no se controlan los ciclos.

PLANTEAMIENTO DEL PROBLEMA Y REPRESENTACIÓN DEL GRAFO

Encontrar un camino de salida para un agente ubicado en la casilla inicial de un laberinto en cuadrícula hasta alcanzar la celda de salida (meta), implementando y comparando el rendimiento de BFS y DFS.

Representación del Grafo

El laberinto se modela como un grafo no dirigido $G = (V, E)$ donde:

- Vértices (V): Celdas transitables de la cuadrícula representadas por sus coordenadas (x, y).
- Aristas (E): Conexiones libres entre celdas adyacentes (Norte, Sur, Este, Oeste) que no contienen muros.

CÓDIGO EN PYTHON (IMPLEMENTACIÓN Y COMPARATIVA)

```
"""
================================================================================
ACTIVIDAD 4: BÚSQUEDA CIEGA - BFS VS DFS EN LABERINTO
================================================================================
"""

from collections import deque
import time

# 1. REPRESENTACIÓN DEL LABERINTO (0 = Camino libre, 1 = Muro/Obstáculo)
laberinto = [
    [0, 1, 0, 0, 0],
    [0, 1, 0, 1, 0],
    [0, 0, 0, 1, 0],
    [1, 1, 0, 0, 0],
    [0, 0, 0, 1, 0]
]
```

```
inicio = (0, 0)

meta = (4, 4)


def obtener_vecinos(nodo, lab):
    x, y = nodo
    filas = len(lab)
    cols = len(lab[0])
    vecinos = []


    # Movimientos cardinales: Norte, Sur, Este, Oeste
    direcciones = [(-1, 0), (1, 0), (0, 1), (0, -1)]
    for dx, dy in direcciones:
        nx, ny = x + dx, y + dy
        if 0 <= nx < filas and 0 <= ny < cols and lab[nx][ny] == 0:
            vecinos.append((nx, ny))
    return vecinos


# 2. IMPLEMENTACIÓN DE BFS (Búsqueda en Anchura)
def busqueda_bfs(lab, start, goal):
    frontera = deque([(start, [start])])
    visitados = {start}
```

```
    nodos_expandidos = 0

    inicio_tiempo = time.time()
    while frontera:
        actual, camino = frontera.popleft()
        nodos_expandidos += 1

        if actual == goal:
            tiempo_ejecucion = time.time() - inicio_tiempo
            return camino, nodos_expandidos, tiempo_ejecucion

        for vecino in obtener_vecinos(actual, lab):
            if vecino not in visitados:
                visitados.add(vecino)
                frontera.append((vecino, camino + [vecino]))
    return None, nodos_expandidos, time.time() - inicio_tiempo

# 3. IMPLEMENTACIÓN DE DFS (Búsqueda en Profundidad)
def busqueda_dfs(lab, start, goal):
    frontera = [(start, [start])]
    visitados = set()
```

```
    nodos_expandidos = 0

    inicio_tiempo = time.time()
    while frontera:
        actual, camino = frontera.pop()

        if actual in visitados:
            continue
        visitados.add(actual)
        nodos_expandidos += 1

        if actual == goal:
            tiempo_ejecucion = time.time() - inicio_tiempo
            return camino, nodos_expandidos, tiempo_ejecucion

        for vecino in obtener_vecinos(actual, lab):
            if vecino not in visitados:
                frontera.append((vecino, camino + [vecino]))
    return None, nodos_expandidos, time.time() - inicio_tiempo
```

```
#
==========================================================================
=============

# EJECUCIÓN Y COMPARATIVA EN CONSOLA

#
==========================================================================
=============

if __name__ == "__main__":

    print("--- RESULTADOS DE BÚSQUEDA EN LABERINTO ---")


    # Ejecutar BFS

    camino_bfs, nodos_bfs, tiempo_bfs = busqueda_bfs(laberinto, inicio, meta)

    print(f"\n[BFS - Anchura]")

    print(f" -> Ruta solución ({len(camino_bfs)} pasos): {camino_bfs}")

    print(f" -> Nodos expandidos: {nodos_bfs}")

    print(f" -> Tiempo de ejecución: {tiempo_bfs:.6f} segundos")


    # Ejecutar DFS

    camino_dfs, nodos_dfs, tiempo_dfs = busqueda_dfs(laberinto, inicio, meta)

    print(f"\n[DFS - Profundidad]")

    print(f" -> Ruta encontrada ({len(camino_dfs)} pasos): {camino_dfs}")

    print(f" -> Nodos expandidos: {nodos_dfs}")

    print(f" -> Tiempo de ejecución: {tiempo_dfs:.6f} segundos")
```

ANÁLISIS DE COMPLEJIDAD TEÓRICA

Complejidad Temporal:

- *BFS:* $O(b^d)$, donde b es el factor de ramificación y d es la profundidad de la solución. Explora todos los nodos del nivel antes de profundizar.
- *DFS:* $O(b^m)$, donde m es la profundidad máxima del espacio de estados. Puede ser catastrófico si m es muy grande o infinito.

Complejidad Espacial:

- *BFS:* $O(b^d)$, ya que almacena todos los nodos de la frontera en la memoria RAM (la cola crece exponencialmente).
- *DFS:* $O(b \cdot m)$, debido a que solo requiere almacenar los nodos de la ruta actual desde la raíz hasta la hoja (muy eficiente en memoria).

Asistente de Inteligencia Artificial (Google Gemini).

COMPARATIVA DE RESULTADOS Y RENDIMIENTO

Métrica de Evaluación	BFS (Anchura)	DFS (Profundidad)
Garantía de Optimalidad	Sí (Encuentra siempre el camino más corto).	No (Encuentra el primer camino viable que halle).
Nodos Expandidos	Tiende a explorar más nodos horizontalmente para asegurar el mínimo costo.	Explora menos nodos si acierta rápido en la rama profunda.
Longitud del Camino Solución	Óptima (mínimo número de movimientos).	Usualmente más largo y subóptimo.
Consumo de Memoria	Mayor (almacena niveles completos en cola).	Menor (almacena únicamente la rama actual).

Asistente de Inteligencia Artificial (Google Gemini).

CONCLUSIONES E IDONEIDAD SEGÚN EL PROBLEMA

Idoneidad de BFS: Es el algoritmo ideal cuando el objetivo principal es encontrar la ruta óptima o más corta en tableros, mapas o laberintos con costos de paso uniformes, siempre que la memoria disponible del sistema sea suficiente para soportar el crecimiento de la frontera.

Idoneidad de DFS: Resulta útil en escenarios donde los recursos de memoria son sumamente limitados o cuando se busca cualquier solución de manera rápida en espacios profundos, sacrificando por completo la optimalidad del recorrido.

FUENTES

Bohm, G. (s. f.). Algoritmos de búsqueda ciega: BFS y DFS (Cap. 3).

Russell, S., & Norvig, P. (2020). Inteligencia artificial: Un enfoque moderno (4ª ed.). Pearson Educación.

Python Software Foundation. (s. f.). Python Language Reference (Versión 3.13) [Software de computadora]. https://www.python.org/

Asistente de Inteligencia Artificial (Google Gemini).